

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Gerência de Memória

Sistemas Operacionais

Bernardo Souza Muniz.

2 de Junho de 2026

Engenharia de Telecomunicações - IFSC-SJ

Sumário

1. Conhecendo a Memória do Sistema	3
2. Informações Detalhadas da memória	3
3. Monitorando Processos	5
4. Criando Consumo Artificial de Memória	6
5. Mapeando a Hierarquia de Memória (Caches e RAM)	7
6. Explorando a Memória Virtual	9
7. Explorando Endereços Lógicos	10
8. Explorando Espaços de Endereçamento	12

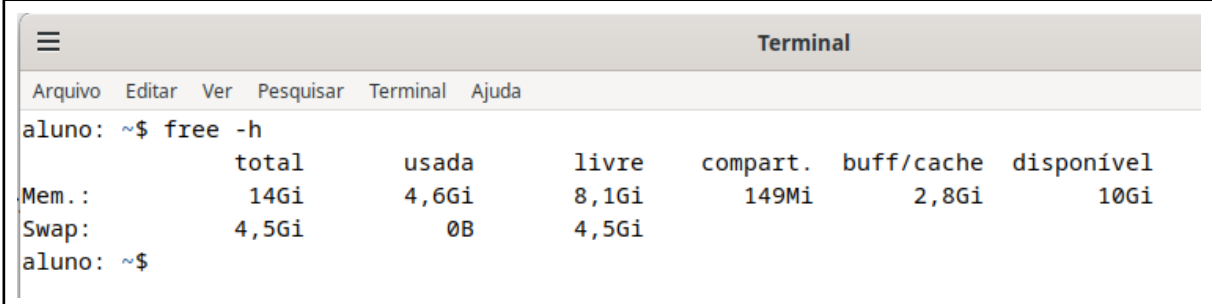
1. Conhecendo a Memória do Sistema

Dentro da máquina local, foi executado o seguinte comando:

```
1 free -h
```

Ao executar o comando supracitado, foi obtido o seguinte resultado:

Figura 1: Elaborada pelo Autor



	total	usada	livre	compart.	buff/cache	disponível
Mem.:	14Gi	4,6Gi	8,1Gi	149Mi	2,8Gi	10Gi
Swap:	4,5Gi	0B	4,5Gi			

Execução do comando free -h

A partir do retorno, foi possível anotar os seguintes valores referentes a utilização da memória do computador:

- Memória RAM total: 14 GBs
- Memória RAM utilizada: 4,6 GBs
- Memória RAM livre: 8,1 GBs
- Espaço total de Swap: 4,5 GBs

O que é memória Swap?

A memória Swap é uma parte do HD ou SSD que é utilizada como extensão da memória RAM do Sistema Operacional. Ela é utilizada quando a memória RAM do computador fica cheia, armazenando temporariamente processos que estão inativos para liberar espaço de utilização para os programas que estão sendo executados no momento, evitando travamentos no sistema operacional

2. Informações Detalhadas da memória

Para realizar o acesso as informações sobre a memória através do sistema de arquivos, foi executado o seguinte comando:

```
1 cat /proc/meminfo
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 2: Elaborada pelo Autor

```
aluno: ~$ cat /proc/meminfo
MemTotal:      15726908 kB
MemFree:       8432356 kB
MemAvailable:  10914648 kB
Buffers:       175696 kB
Cached:        2632500 kB
SwapCached:    0 kB
Active:        1430736 kB
Inactive:      4813728 kB
Active(anon):  7992 kB
Inactive(anon): 3571456 kB
Active(file):  1422744 kB
Inactive(file): 1242272 kB
Unevictable:   15840 kB
Mlocked:       15840 kB
SwapTotal:     4729852 kB
SwapFree:      4729852 kB
```

Execução do comando `cat /proc/meminfo`

Com isso, foi possível anotar as seguintes informações sobre a memória do sistema operacional:

- **MemTotal:** 15726908 kB = 15.72 Gbps
- **MemFree:** 8432356 kB = 8.43 Gbps
- **Buffers:** 175696 kB = 175.696 Mbps
- **Cached:** 2632500 kB = 2.6325 Gbps
- **Swap Total:** 4729852 kB = 4.72 Gbps
- **SwapFree:** 4729852 kB = 4.72 Gbps

O que representa a memória cache?

A memória cache é um tipo de armazenamento aleatório de dados que é utilizado para armazenar informações que são acessadas frequentemente para obter um tempo de resposta mais rápido em comparação com outros tipos de armazenamentos. A medida que mais dados vão sendo armazenados no cache, menos tempo se perde acessando os dados da memória RAM.

Por que o Linux utiliza parte da RAM como cache?

O Linux utiliza parte da memória RAM livre como cache para aumentar o desempenho do sistema. Em vez de deixar a memória que não está sendo usada sem utilidade, o sistema operacional armazena nela arquivos e dados acessados recentemente, para que quando forem novamente acessados, o tempo de resposta seja muito menor.

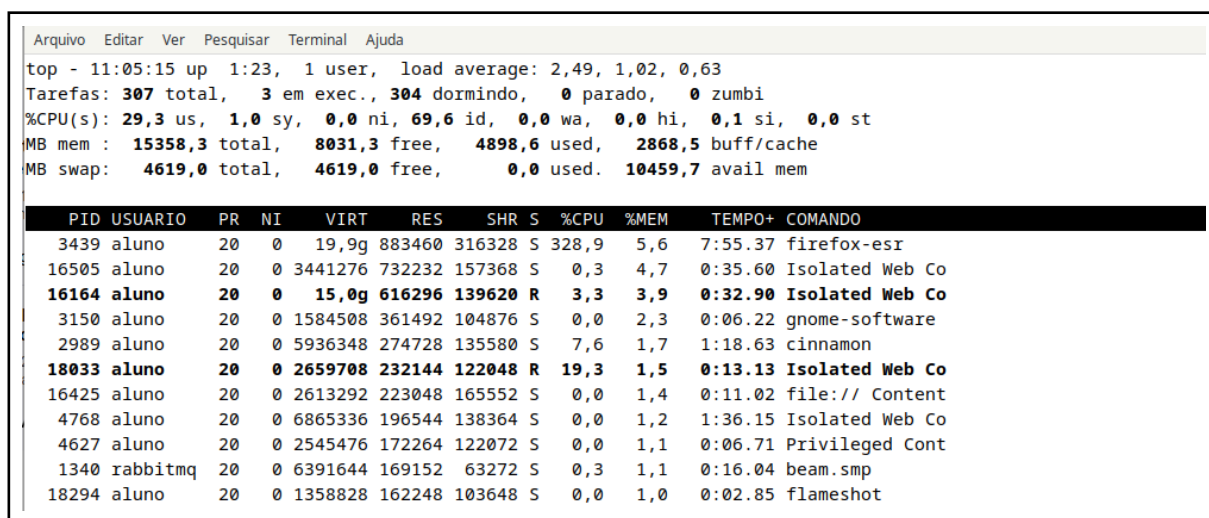
3. Monitorando Processos

Foi executado o seguinte comando para observar o consume de memória dos processos em execução:

```
1 top
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 3: Elaborada pelo Autor



PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPO+	COMANDO
3439	aluno	20	0	19,9g	883460	316328	S	328,9	5,6	7:55.37	firefox-esr
16505	aluno	20	0	3441276	732232	157368	S	0,3	4,7	0:35.60	Isolated Web Co
16164	aluno	20	0	15,0g	616296	139620	R	3,3	3,9	0:32.90	Isolated Web Co
3150	aluno	20	0	1584508	361492	104876	S	0,0	2,3	0:06.22	gnome-software
2989	aluno	20	0	5936348	274728	135580	S	7,6	1,7	1:18.63	cinnamon
18033	aluno	20	0	2659708	232144	122048	R	19,3	1,5	0:13.13	Isolated Web Co
16425	aluno	20	0	2613292	223048	165552	S	0,0	1,4	0:11.02	file:// Content
4768	aluno	20	0	6865336	196544	138364	S	0,0	1,2	1:36.15	Isolated Web Co
4627	aluno	20	0	2545476	172264	122072	S	0,0	1,1	0:06.71	Privileged Cont
1340	rabbitmq	20	0	6391644	169152	63272	S	0,3	1,1	0:16.04	beam.smp
18294	aluno	20	0	1358828	162248	103648	S	0,0	1,0	0:02.85	flameshot

Execução do comando top

Identifique os cinco processos que mais consomem memória:

Os processos que mais consomem memória são:

- PID 3439: firefox-esr - MEM: 5,6%
- PID 16505: Isolated Web Co - MEM: 4,7%
- PID 16164: Isolated Web Co - MEM: 3,9%
- PID 3150: gnome-software - MEM: 2,3%
- PID 2989: cinnamon - MEM: 1,7%

Responda:

(a) Qual processo consome mais memória?

O processo que mais consome memória é o firefox-esr, utilizando 5,6% da memória disponível.

(b) Esse processo pertence ao sistema operacional ou ao usuário?

Pertence ao usuário, conforme indicado na coluna USUARIO. Como se trata de um navegador, faz parte de uma aplicação para usuário e não um serviço essencial do sistema operacional.

4. Criando Consumo Artificial de Memória

Para criar um programa que consome boa parte da memória RAM, foi utilizado os seguintes comandos:

1) Foi aberto o interpretador python utilizando o terminal do computador:

```
1 python3
```

2) Foi digitado o seguinte comando:

```
1 dados = [0] * 1000000000
```

3) Foi um terminal para monitorar o consumo da memória RAM:

```
1 free -h
```

Figura 4: Elaborada pelo Autor

```
aluno: ~$ free -h
              total        usada        livre      compart.  buff/cache  disponível
Mem.:          14Gi         12Gi         263Mi        127Mi         2,9Gi         2,7Gi
Swap:          4,5Gi           0B         4,5Gi
aluno: ~$
```

Execução do comando free -h

4) Em seguida foi listado os processos do sistema com o seguinte comando:

```
1 top
```

Figura 5: Elaborada pelo Autor

```
top - 11:09:14 up 1:27, 1 user, load average: 0,65, 0,83, 0,65
Tarefas: 312 total, 2 em exec., 310 dormindo, 0 parado, 0 zumbi
%CPU(s): 2,4 us, 0,9 sy, 0,0 ni, 96,6 id, 0,0 wa, 0,0 hi, 0,1 si, 0,0 st
MB mem : 15358,3 total, 256,6 free, 12582,9 used, 2961,7 buff/cache
MB swap: 4619,0 total, 4619,0 free, 0,0 used, 2775,4 avail mem

  PID USUARIO PR NI VIRT RES SHR S %CPU %MEM TEMPO+ COMANDO
  26609 aluno 20 0 7829116 7,5g 6112 S 0,0 49,7 0:01.00 python3
  3439 aluno 20 0 19,9g 871192 326676 S 4,3 5,5 9:43.12 firefox-esr
  16505 aluno 20 0 3435156 735408 150224 S 0,7 4,7 0:37.38 Isolated Web Co
  16164 aluno 20 0 15,0g 616228 139620 S 2,6 3,9 0:33.69 Isolated Web Co
  3150 aluno 20 0 1584508 361492 104876 S 0,0 2,3 0:06.27 gnome-software
  2989 aluno 20 0 5938624 267776 135628 S 3,6 1,7 1:28.58 cinnamon
  18033 aluno 20 0 2645104 227628 121912 S 0,3 1,4 0:19.37 Isolated Web Co
  16425 aluno 20 0 2614372 225840 167000 S 0,0 1,4 0:13.16 file:/// Content
  4768 aluno 20 0 6865336 197496 138364 R 0,7 1,3 1:37.52 Isolated Web Co
  18294 aluno 20 0 1370600 170304 109960 S 0,0 1,1 0:06.49 flameshot
```

Execução do comando free -h

Responda:

(a) Quanto a memória utilizada aumentou?

De acordo com a figura 4, a memória RAM usado passou a ser de 12 GB. De acordo com a figura 1, que lista a memória utilizada antes do código ser executado, a memória era de 4,6 GB, resultando em um aumento de 7,4 GB de consumo.

(b) O processo Python apareceu entre os maiores consumidores de memória?

Sim. De acordo com a listagem de processos da figura 5, é possível observar que o processo Python consumiu 49,7% da memória RAM do sistema, sendo listado como o que mais consome memória RAM.

(c) O que aconteceu com a quantidade de memória livre?

Foi diminuída consideravelmente. De acordo com a figura 5, o sistema operacional possui 263 MB livres. Em comparação com a figura 1, que lista a memória livre antes do código ser utilizado, é possível observar que havia 8,1 GB livres.

5. Mapeando a Hierarquia de Memória (Caches e RAM)

Foi utilizado o seguinte comando para listar informações sobre os níveis de cache instalados no processador:

```
1 lscpu | grep -i "cache"
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 6: Elaborada pelo Autor

```
aluno: ~$ lscpu | grep -i "cache"
cache de L1d:                192 KiB (6 instâncias)
cache de L1i:                192 KiB (6 instâncias)
cache de L2:                 3 MiB (6 instâncias)
cache de L3:                 16 MiB (1 instância)
aluno: ~$ █
```

Execução do comando `lscpu | grep -i "cache"`

Para entender a geometria do cache de L1 de dados, foi utilizado os seguintes comandos:

```
1 cat /sys/devices/system/cpu/cpu0/cache/index0/size
2
3 cat /sys/devices/system/cpu/cpu0/cache/index0/type
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 7: Elaborada pelo Autor

```
aluno: ~$ cat /sys/devices/system/cpu/cpu0/cache/index0/size
32K
aluno: ~$ cat /sys/devices/system/cpu/cpu0/cache/index0/type
Data
aluno: ~$
```

Execução dos comandos de validação de cache L1

Qual é o tamanho dos caches L1, L2 e L3 da sua máquina? Explique, com base na teoria da pirâmide de memória, por que o cache L1 é drasticamente menor do que a memória RAM instalada.

O tamanho da memória cache L1 é drasticamente menor que a da memória RAM devido ao fato de que ela é utilizada para armazenar temporariamente os dados e instruções que são acessados frequentemente, o que normalmente ocupa um espaço muito pequeno. Diferentemente da memória RAM, que necessita armazenar programas e dados que ocupam mais espaço dentro do sistema operacional.

Analisando o retorno do comando da figura 6, foi possível observar os seguintes valores referentes as memórias de cache:

- **L1:** 192 KB
- **L2:** 3 MB
- **L3:** 16 MB

6. Explorando a Memória Virtual

Foi executado o seguinte comando para verificar as estatísticas relacionadas ao uso da memória virtual:

```
1 vmstat 2
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 8: Elaborada pelo Autor

```
aluno: ~$ vmstat 2
```

procs		-----memória-----				---swap--		----e/s----		-sistema-		-----cpu-----				
r	b	swpd	livre	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
2	0	0	8138124	185176	2849904	0	0	35	23	172	260	2	0	98	0	0
1	0	0	8138124	185176	2849904	0	0	0	20	513	697	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	22	663	976	0	0	99	0	0
2	0	0	8138124	185176	2849904	0	0	0	2	477	657	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	557	775	0	0	99	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	5103	7567	3	1	96	0	0
1	0	0	8138124	185176	2849904	0	0	0	344	5117	8077	2	1	97	0	0

procs		-----memória-----				---swap--		----e/s----		-sistema-		-----cpu-----				
r	b	swpd	livre	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
2	0	0	8138124	185176	2849904	0	0	0	0	2000	2826	1	0	99	0	0
2	0	0	8138124	185176	2849904	0	0	0	12	505	666	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	1053	1428	2	0	97	0	0
1	0	0	8138124	185176	2849904	0	0	0	2	600	864	1	0	99	0	0
1	0	0	8138124	185176	2849904	0	0	0	58	604	844	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	480	615	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	94	485	661	0	0	99	0	0

procs		-----memória-----				---swap--		----e/s----		-sistema-		-----cpu-----				
r	b	swpd	livre	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
1	0	0	8138124	185176	2849904	0	0	0	50	636	856	0	0	99	0	0
2	0	0	8138124	185176	2849904	0	0	0	0	698	1007	0	0	99	0	0
1	0	0	8138124	185176	2849904	0	0	0	6	932	1383	1	0	99	0	0
1	0	0	8138124	185176	2849904	0	0	0	98	1469	2329	1	0	98	0	0
2	0	0	8138124	185176	2849904	0	0	0	6	534	767	0	0	99	0	0
2	0	0	8138124	185176	2849904	0	0	0	0	427	611	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	30	603	889	0	0	100	0	0

procs		-----memória-----				---swap--		----e/s----		-sistema-		-----cpu-----				
r	b	swpd	livre	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
2	0	0	8138124	185176	2849904	0	0	0	0	659	975	1	0	99	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	549	791	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	0	510	745	0	0	100	0	0
1	0	0	8138124	185176	2849904	0	0	0	20	1273	2457	1	0	99	0	0

Execução do comando vmstat 2

(a) Houve movimentação entre RAM e swap?

Não. Os campos si (swap in) e so (swap out) estão zerados (0) em todo o histórico de observação. Isso significa que nenhum dado foi lido do disco para a RAM e nenhum dado foi gravado da RAM para a área de swap durante o período observado.

(b) O sistema apresentou sinais de falta de memória?

Não. O sistema está trabalhando com total folga de memória, pois a coluna de **memória livre** permanece constante e sem realizar swap de memória RAM.

7. Explorando Endereços Lógicos

1) Foi criado o arquivo `endereco.c` com o seguinte código:

```
1  #include <stdio.h>
2  int global = 10;
3  int main() {
4  int local = 20;
5  printf("Endereco da variavel global: %p\n", &global);
6  printf("Endereco da variavel local : %p\n", &local);
7  getchar();
8  return 0;
9  }
```

Ao compilar e executar o código, foi obtido o seguinte retorno:

Figura 9: Elaborada pelo Autor

```
aluno: ~$ gcc endereco.c -o endereco
aluno: ~$ ./endereco
Endereco da variavel global: 0x562e46d67020
Endereco da variavel local : 0x7ffee098428c
█
```

Retorno do código `endereco.c`

Foi mantido o programa em execução e aberto outro terminal para obter o PID do processo do código com o seguinte comando:

```
1  ps aux | grep endereco
```

Ao executar o comando, foi obtido o seguinte PID:

Figura 10: Elaborada pelo Autor

```
aluno: ~$ ps aux | grep endereco
aluno      28427  0.0  0.0   2468   872 pts/0    S+   11:19   0:00  ./endereco
aluno      28536  0.0  0.0   6432  2096 pts/1    S+   11:19   0:00  grep endereco
aluno: ~$ █
```

PID do processo

Por fim, foi visualizado o mapa de memória do processo com o seguinte comando:

```
1  cat /proc/28427/maps
```

Ao executar o comando, foi obtido o seguinte resultado:

Figura 11: Elaborada pelo Autor

```
aluno: ~$ ps aux | grep endereco
aluno      28427  0.0  0.0  2468   872 pts/0    S+   11:19   0:00 ./endereco
aluno      28684  0.0  0.0  6432  2148 pts/1    S+   11:21   0:00 grep endereco
aluno: ~$ cat /proc/28427/maps
562e46d63000-562e46d64000 r--p 00000000 103:06 809510      /home/aluno/endereco
562e46d64000-562e46d65000 r-xp 00001000 103:06 809510      /home/aluno/endereco
562e46d65000-562e46d66000 r--p 00002000 103:06 809510      /home/aluno/endereco
562e46d66000-562e46d67000 r--p 00002000 103:06 809510      /home/aluno/endereco
562e46d67000-562e46d68000 rw-p 00003000 103:06 809510      /home/aluno/endereco
562e7bde5000-562e7be06000 rw-p 00000000 00:00 0          [heap]
7fb113acd000-7fb113ad0000 rw-p 00000000 00:00 0
7fb113ad0000-7fb113af6000 r--p 00000000 103:06 55159      /usr/lib/x86_64-linux-gnu/libc.so.6
7fb113af6000-7fb113c4c000 r-xp 00026000 103:06 55159      /usr/lib/x86_64-linux-gnu/libc.so.6
7fb113c4c000-7fb113c9f000 r--p 0017c000 103:06 55159      /usr/lib/x86_64-linux-gnu/libc.so.6
7fb113c9f000-7fb113ca3000 r--p 001cf000 103:06 55159      /usr/lib/x86_64-linux-gnu/libc.so.6
7fb113ca3000-7fb113ca5000 rw-p 001d3000 103:06 55159      /usr/lib/x86_64-linux-gnu/libc.so.6
7fb113ca5000-7fb113cb2000 rw-p 00000000 00:00 0
7fb113cd0000-7fb113cdf000 rw-p 00000000 00:00 0
7fb113cdf000-7fb113ce0000 r--p 00000000 103:06 55153      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fb113ce0000-7fb113d06000 r-xp 00001000 103:06 55153      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fb113d06000-7fb113d10000 r--p 00027000 103:06 55153      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fb113d10000-7fb113d12000 r--p 00031000 103:06 55153      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fb113d12000-7fb113d14000 rw-p 00033000 103:06 55153      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fffee0965000-7fffee0986000 rw-p 00000000 00:00 0          [stack]
7fffee0986000-7fffee098f000 r--p 00000000 00:00 0          [vvar]
7fffee098f000-7fffee0991000 r-xp 00000000 00:00 0          [vdso]
aluno: ~$
```

Retorno do comando cat /proc/28427/maps

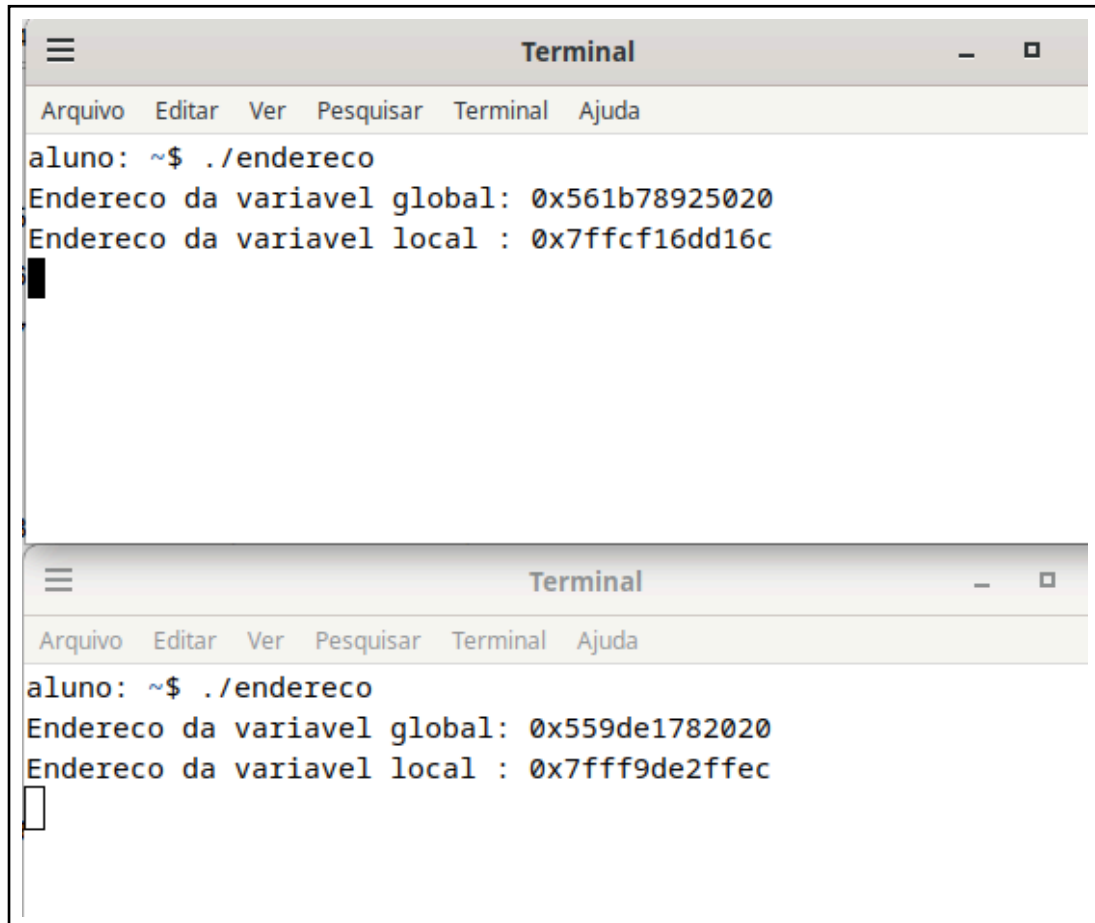
Ao visualizar o mapa de memória, é possível observar as seguintes regiões:

- **Código do programa:** 562e46d63000 - 562e46d68000
- **Heap:** 562e7bde5000 - 562e7be06000
- **Stack:** 7fffee0965000 - 7fffee0986000
- **Bibliotecas compartilhadas:** 7fb113acd000 - 7fb113d14000

8. Explorando Espaços de Endereçamento

Foi aberto dois terminais e executado de maneira simultânea o programa `endereco.c` do exercício anterior.

Figura 12: Elaborada pelo Autor



```
Terminal
Arquivo Editar Ver Pesquisar Terminal Ajuda
aluno: ~$ ./endereco
Endereco da variavel global: 0x561b78925020
Endereco da variavel local : 0x7ffc16dd16c

Terminal
Arquivo Editar Ver Pesquisar Terminal Ajuda
aluno: ~$ ./endereco
Endereco da variavel global: 0x559de1782020
Endereco da variavel local : 0x7fff9de2ffec
```

Execução simultânea do programa `endereco.c`

- É possível verificar que o endereço da variável global é o mesmo para ambos os programas. Somente o endereço da variável local é alterado.
- Isso acontece porque o armazenamento da variável global é feito de maneira estática enquanto o da variável local é feita de maneira automática.

Ao buscar o PID de cada processo em execução do código, foi obtido o seguinte retorno

Figura 13: Elaborada pelo Autor

```
aluno: ~$ ps aux | grep endereco
aluno      28961  0.0  0.0   2468    888 pts/1    S+   11:22   0:00 ./endereco
aluno      28964  0.0  0.0   2468    900 pts/0    S+   11:22   0:00 ./endereco
aluno      29156  0.0  0.0   6432   2196 pts/2    S+   11:24   0:00 grep endereco
```

PIDs dos processos do código `endereco.c`

Fazendo a busca pelo mapa de memória de cada processo:

Figura 14: Elaborada pelo Autor

```
aluno: ~$ cat /proc/28961/maps
561b78921000-561b78922000 r--p 00000000 103:06 809510 /home/aluno/endereco
561b78922000-561b78923000 r-xp 00001000 103:06 809510 /home/aluno/endereco
561b78923000-561b78924000 r--p 00002000 103:06 809510 /home/aluno/endereco
561b78924000-561b78925000 r--p 00002000 103:06 809510 /home/aluno/endereco
561b78925000-561b78926000 rw-p 00003000 103:06 809510 /home/aluno/endereco
561b8a74a000-561b8a76b000 rw-p 00000000 00:00 0 [heap]
7f7608029000-7f760802c000 rw-p 00000000 00:00 0
7f760802c000-7f7608052000 r--p 00000000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f7608052000-7f76081a8000 r-xp 00026000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f76081a8000-7f76081fb000 r--p 0017c000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f76081fb000-7f76081ff000 r--p 001cf000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f76081ff000-7f7608201000 rw-p 001d3000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f7608201000-7f760820e000 rw-p 00000000 00:00 0
7f760820e000-7f760823b000 rw-p 00000000 00:00 0
7f760823b000-7f760823c000 r--p 00000000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f760823c000-7f7608262000 r-xp 00001000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f7608262000-7f760826c000 r--p 00027000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f760826c000-7f760826e000 r--p 00031000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f760826e000-7f7608270000 rw-p 00033000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffcf16bf000-7ffcf16e0000 rw-p 00000000 00:00 0 [stack]
7ffcf16f2000-7ffcf16f6000 r--p 00000000 00:00 0 [vvar]
7ffcf16f6000-7ffcf16f8000 r-xp 00000000 00:00 0 [vdso]
```

Mapa de memória do processo com PID 28961

Figura 15: Elaborada pelo Autor

```
aluno: ~$ cat /proc/28964/maps
559de177e000-559de177f000 r--p 00000000 103:06 809510 /home/aluno/endereco
559de177f000-559de1780000 r-xp 00001000 103:06 809510 /home/aluno/endereco
559de1780000-559de1781000 r--p 00002000 103:06 809510 /home/aluno/endereco
559de1781000-559de1782000 r--p 00002000 103:06 809510 /home/aluno/endereco
559de1782000-559de1783000 rw-p 00003000 103:06 809510 /home/aluno/endereco
559deef0a000-559deef2b000 rw-p 00000000 00:00 0 [heap]
7f6880ff6000-7f6880ff9000 rw-p 00000000 00:00 0
7f6880ff9000-7f688101f000 r--p 00000000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f688101f000-7f6881175000 r-xp 00026000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f6881175000-7f68811c8000 r--p 0017c000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f68811c8000-7f68811cc000 r--p 001cf000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f68811cc000-7f68811ce000 rw-p 001d3000 103:06 55159 /usr/lib/x86_64-linux-gnu/libc.so.6
7f68811ce000-7f68811db000 rw-p 00000000 00:00 0
7f6881206000-7f6881208000 rw-p 00000000 00:00 0
7f6881208000-7f6881209000 r--p 00000000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f6881209000-7f688122f000 r-xp 00001000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f688122f000-7f6881239000 r--p 00027000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f6881239000-7f688123b000 r--p 00031000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f688123b000-7f688123d000 rw-p 00033000 103:06 55153 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff9de11000-7fff9de32000 rw-p 00000000 00:00 0 [stack]
7fff9de4d000-7fff9de51000 r--p 00000000 00:00 0 [vvar]
7fff9de51000-7fff9de53000 r-xp 00000000 00:00 0 [vdso]
aluno: ~$
```

Mapa de memória do processo com PID 28964

Com isso, foi possível verificar que cada processo possui seu próprio espaço de endereçamento lógico, pois possuem o mesmo mapa de memória para PIDs diferentes.